# Seekra — Open-Source Components & Security Vulnerability Management

**Prepared for:** Client review **Date:** 2026-08-02 **Scope:** All third-party (open-source) technologies used in the Seekra platform, and the process by which future vulnerabilities or bugs in them are handled and remediated.

---

## 1. Technology inventory (all open-source)

| Layer | Component | License | Role in Seekra |
|---|---|---|---|
| Frontend framework | React 18 / Next.js 14 | MIT | Bilingual EN/AR UI, static export |
| Language (UI) | TypeScript | Apache-2.0 | Type-safe frontend |
| Styling | Tailwind CSS | MIT | Design system |
| PDF rendering | PDF.js 4.10 (Mozilla) | Apache-2.0 | In-browser document viewer |
| API framework | FastAPI | MIT | REST API, auth, search, admin |
| Language (API) | Python 3.12 | PSF | Backend & workers |
| Task queue | Celery 5 | BSD-3 | Async ingest pipeline |
| Broker / cache | Redis 7 | RSALv2/SSPLv1 (open) | Queue, cache, login throttling |
| Database | PostgreSQL 16 + pgvector | PostgreSQL / PostgreSQL | Records + semantic vector search |
| Object storage | MinIO | AGPLv3 | S3-compatible file storage |
| OCR | Tesseract | Apache-2.0 | Scanned-document text extraction |
| Edge / TLS | Nginx 1.31 | BSD-2 | HTTPS, security headers, proxy |
| Container runtime | Docker / Docker Compose | Apache-2.0 | Packaging & orchestration |
| Host OS | Ubuntu LTS | Canonical (open) | AWS EC2 host |

No proprietary or closed-source dependency ships in the platform.

---

## 2. The client's question

> "Since all components are open-source, how are security vulnerabilities or bugs that arise in the future handled, and how are they remediated?"

---

## 3. Expert answer — step by step

### Step 1 — "Open-source" does not mean "unsupported"

Every component in the stack is a **mainstream, actively maintained project** with a formal security process:

- **PostgreSQL, Nginx, Redis, Python, FastAPI, Next.js** all run dedicated security teams, publish CVEs, and ship patched releases on predictable cadences (e.g., PostgreSQL quarterly minor releases; Nginx mainline/stable branches).
- Open-source actually has a **structural advantage**: code is publicly auditable, vulnerabilities are disclosed publicly (CVE database) rather than hidden, and fixes are usually available *faster* than in closed-source vendor cycles.

### Step 2 — Detection: how we learn about a vulnerability

1. **CVE / NVD feeds** — every public vulnerability gets a CVE identifier scored by CVSS severity (Critical / High / Medium / Low).
2. **GitHub security advisories + Dependabot** — the Seekra repository is on GitHub (`rahmirzconsulting-ai/genoa-platform`); Dependabot continuously scans `package.json`, `requirements.txt`, and Docker base images and opens alert/PR when a dependency version we use is affected.
3. **Vendor security lists** — PostgreSQL-announce, Nginx security advisories, Redis security page, Python security announcements.
4. **Container image scanning** — base images (`python:3.12-slim`, `nginx`, `postgres:16`, `redis:7`, `minio`) are scanned (e.g., `docker scout` / Trivy) so OS-level CVEs inside containers surface too.

### Step 3 — Triage: how we decide urgency

Each alert is assessed on three axes:

| Factor | Question | Example in Seekra context |
|---|---|---|
| Severity | CVSS score? | Critical ( 9.0) → act within 24–72h |
| Reachability | Is the vulnerable code path actually used/exposed? | A PDF.js parser bug matters (users render PDFs); a bug in an unused Next.js server-action path does not apply to our static export |
| Exposure | Internet-facing or internal? | Nginx/FastAPI are edge-exposed → highest priority; Redis/PostgreSQL/MinIO are on a private Docker network, not reachable from the internet → lower practical risk |

This avoids panic-patching noise while guaranteeing real risks are fixed fast.

**Step 4 — Remediation: the concrete fix path**

Because Seekra is **fully containerized with pinned dependencies**, remediation is mechanical and low-risk:

1. **Update the pinned version** — bump the affected package/image tag (e.g., `pdfjs-dist 4.10.38 → patched`, `postgres:16.x → 16.y`, `nginx:1.31.x → 1.31.y`).
2. **Rebuild & redeploy** — the same deploy workflow used for every change:
   - backend/edge: `docker compose up -d --build <service>` (zero-downtime for stateless services)
   - frontend: `npm run build` + Nginx restart
3. **Verify** — live smoke checks (auth flow, search, viewer) plus existing audit trail confirms no regression.
4. **Data is never at risk during patching** — PostgreSQL data, MinIO objects, and Redis are in persistent volumes; container replacement does not touch data.

Typical remediation time: **minutes to hours** for minor/major patches, not days — because no code changes are needed, only version bumps and rebuilds.

**Step 5 — If a fix is not yet available (zero-day window)**

Defense-in-depth already limits blast radius while waiting for an upstream patch:

- **Edge hardening (Step 19):** HSTS, X-Frame-Options DENY, nosniff, Referrer-Policy, Permissions-Policy — block whole attack classes (clickjacking, MIME sniffing, XSS amplification).
- **Network isolation:** Redis, PostgreSQL, and MinIO are only reachable inside the Docker network; the internet can only touch Nginx → FastAPI.
- **Auth throttling & lockout:** brute-force is capped (5 failures → 15-minute lock, IP-level limits, admin unlock controls).
- **RBAC + audit trail:** every security-relevant action is logged (`SECURITY_LOGIN_*`, `PASSWORD_RESET`, file/collection events), so any incident is forensically reconstructable.
- **Workaround configs:** most zero-days can be mitigated at Nginx/FastAPI level (blocking a path, disabling a feature flag) within minutes, before the official patch lands.

**Step 6 — Ongoing maintenance plan (what we commit to)**

| Cadence | Activity |
|---|---|
| Continuous | Dependabot + CVE monitoring on the repo |
| Monthly | Review advisories; apply low-risk patch releases |
| Quarterly | Planned maintenance window: refresh all base images & dependencies, rebuild, regression-test |
| On Critical CVE | Emergency patch within 24–72h using the standard rebuild workflow |
| Annual | Major-version upgrades (e.g., Next.js 14 → 15, PostgreSQL 16 → 17) with full staging test |

**Step 7 — Real precedent inside this project**

This process is not theoretical — it has already operated on Seekra:

- **PDF.js:** the legacy v3 viewer line reached end-of-life with unpatched parser issues. It was upgraded to the maintained v4.10 line (Step 21), with the module-worker MIME issue identified and remediated the same day.
- **Identity hardening (Step 19):** login throttling, account lockout, and five HTTP security headers were added proactively, ahead of any incident.
- **Admin account controls (Step 20):** password reset and account-unlock remediation tooling with full audit — the operational half of incident response.

---

## 4. One-paragraph summary for the client

> Every component in Seekra is a mainstream, actively maintained open-source project with a formal security disclosure process. Vulnerabilities are detected automatically via CVE feeds and GitHub Dependabot, triaged by severity and actual exposure in our architecture, and remediated by bumping pinned versions and rebuilding containers — typically within hours, with zero data risk since all state lives in persistent volumes. For the window before an official patch exists, the platform's defense-in-depth (TLS + security headers at the edge, network-isolated data services, RBAC, login throttling, and a complete audit trail) contains the risk. This monitoring-and-patch loop is a standing operational commitment, and it has already been exercised in production on this platform.